

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



HỌC PHẦN: XỬ LÝ NGÔN NGỮ TỰ NHIÊN

BÁO CÁO ĐỒ ÁN

ĐỀ TÀI:

**DỊCH MÁY ANH-PHÁP / ANH-ĐỨC VỚI MÔ HÌNH
ENCODER-DECODER LSTM**

Giảng viên hướng dẫn:

PGS.TS: Nguyễn Tuấn Đăng

Nhóm sinh viên thực hiện

Đặng Nhật Đức - 3123410082

TP. Hồ Chí Minh, tháng năm 2025

MỤC LỤC

I.GIỚI THIỆU	
1.1. Bối cảnh	
1.2. Vấn đề nghiên cứu và mục tiêu đề án	
II.NGHIÊN CỨU LIÊN QUAN	
2.1. Mô hình Sequence-to-Sequence	
2.2. Cơ chế Attention và các cải tiến	
2.3. Đánh giá chất lượng dịch máy (BLEU Score)	
III.TỔNG QUAN DỰ ÁN	
3.1. Bộ dữ liệu sử dụng (Multi30K)	
3.2. Tổ chức mã nguồn	
3.3. Công nghệ sử dụng	
IV.KIẾN TRÚC MÔ HÌNH VÀ CƠ SỞ LÝ THUYẾT	
4.1. Tổng quan kiến trúc Seq2Seq	
4.2. Kiến trúc Encoder–Decoder LSTM	
4.3. Sơ đồ kiến trúc mô hình Encoder–Decoder LSTM	
4.4. Công thức toán học của LSTM	
IV. CHI TIẾT TRIỂN KHAI VÀ KỸ THUẬT TỐI ƯU	
5.1. Tiền xử lý dữ liệu và Tokenization	
5.2. Xây dựng DataLoader và kỹ thuật Padding	
5.3. Thiết kế kiến trúc mô hình trong PyTorch	
5.4. Chiến lược huấn luyện và kỹ thuật tối ưu	
VI. QUÁ TRÌNH HUẤN LUYỆN.....	
6.1. Phân tích quá trình huấn luyện	
6.2. Kết quả đánh giá định lượng (BLEU Score)	
6.3. Phân tích định tính qua các ví dụ dịch mẫu	
VII. PHÂN TÍCH LỖI	
7.1. Phân tích lỗi qua các ví dụ dịch	
7.2. Tổng kết các dạng lỗi phổ biến	
VIII. KẾT LUẬN VÀ ĐỀ XUẤT CẢI TIẾN	
IX. PHỤ LỤC	

Báo cáo Mô hình Dịch máy Anh–Pháp sử dụng LSTM

Encoder–Decoder

Giới thiệu

1.1 Bối cảnh

Dịch máy (Machine Translation – MT) là một trong những bài toán khó và quan trọng trong lĩnh vực Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing – NLP). Trải qua các giai đoạn phát triển từ các hệ thống dựa trên luật (Rule-Based MT) và phương pháp thống kê (Statistical MT), dịch máy đã có bước tiến vượt bậc với sự ra đời của Dịch Máy Nơ-ron (Neural Machine Translation – NMT) nhờ sự phát triển của Deep Learning. Trong đó, kiến trúc Sequence-to-Sequence (Seq2Seq) được xem là nền tảng cốt lõi, cho phép mô hình học trực tiếp mối quan hệ giữa câu nguồn và câu đích, từ đó tạo ra các bản dịch tự nhiên và trôi chảy hơn. Dự án này nhằm mục tiêu giúp sinh viên hiểu rõ và tự xây dựng một hệ thống NMT cơ bản từ đầu.

1.2 Vấn đề nghiên cứu

Mục tiêu của đồ án là xây dựng một mô hình NMT dịch từ tiếng Anh sang tiếng Pháp bắt đầu từ các thành phần cơ bản. Nhóm lựa chọn kiến trúc Sequence-to-Sequence sử dụng mạng LSTM (Long Short-Term Memory) theo đề xuất của Sutskever et al. (2014). Thách thức chính của bài toán nằm ở việc thiết kế bộ mã hóa (Encoder) và bộ giải mã (Decoder) sao cho có thể nén và tái tạo thông tin ngôn ngữ một cách hiệu quả, đặc biệt trong bối cảnh mô hình không sử dụng cơ chế Attention.

Trong dự án, tôi tập trung vào các nhiệm vụ chính sau:

- Triển khai đầy đủ mô hình Encoder–Decoder LSTM hai tầng bằng **PyTorch**, bao gồm hàm truyền xuôi, lan truyền ngược, và tối ưu hóa.
- Thực hiện tiền xử lý và vector hóa dữ liệu **Multi30K** (Anh–Pháp), sử dụng công cụ **spaCy** để tách từ và xây dựng từ điển.
- Huấn luyện mô hình với tham số (số epoch, learning rate, dropout, v.v.) phù hợp, sử dụng *teacher forcing* và *gradient clipping* nhằm cải thiện hội tụ.

- Đánh giá hiệu suất bằng điểm **BLEU** trên tập kiểm thử.
- Phân tích định tính các lỗi chính trong bản dịch mẫu (thiếu từ, thêm từ, sai ngữ pháp) và đề xuất hướng cải tiến (ví dụ: tích hợp **cơ chế Attention**, **Beam Search** và tăng quy mô hoặc sử dụng mạng Transformer).

II. NGHIÊN CỨU LIÊN QUAN

2.1 Sequence-to-Sequence Learning

Kiến trúc **Sequence-to-Sequence (Seq2Seq)** được giới thiệu bởi Sutskever et al. (2014) cho bài toán dịch máy. Mô hình này sử dụng hai mạng hồi quy (RNNs), cụ thể là LSTM: một mạng đóng vai trò **Encoder** để mã hóa câu nguồn và một mạng **Decoder** để giải mã thành câu đích.

Đóng góp chính:

- **Vector ngữ cảnh cố định:** Encoder nén toàn bộ thông tin của câu nguồn (chuỗi đầu vào) vào một vector trạng thái cuối cùng có kích thước cố định (hidden state và cell state cuối cùng của LSTM). Vector này sau đó được sử dụng làm trạng thái khởi tạo cho Decoder.
- **LSTM:** Được sử dụng thay vì RNN truyền thống để giải quyết vấn đề đạo hàm biến mất (vanishing gradient), giúp mô hình nắm bắt được các phụ thuộc dài hạn tốt hơn trong chuỗi văn bản.
- **Teacher Forcing:** Kỹ thuật được sử dụng trong quá trình huấn luyện Decoder, trong đó đầu vào của bước thời gian hiện tại là token tham chiếu (ground truth) của bước thời gian trước đó thay vì token do mô hình dự đoán. Kỹ thuật này giúp ổn định quá trình học và hội tụ nhanh hơn.

Hạn chế:

Tuy nhiên, điểm yếu của kiến trúc LSTM encoder–decoder cơ bản là “giật cổ chai” ngữ cảnh (context vector bottleneck): mọi thông tin ngữ nghĩa của câu nguồn đều bị nén vào một vector cố định. Để khắc phục, Bahdanau và cộng sự (2014) đã đề xuất cơ chế *attention*, cho phép mô hình tính toán một ngữ cảnh riêng

biệt cho mỗi từ đích dự đoán bằng cách cân nhắc các trạng thái ẩn của encoder. Theo Bahdanau, cơ chế này giúp “trọng tâm” vào những phần liên quan của câu nguồn khi giải mã, từ đó cải thiện đáng kể chất lượng dịch. Luong et al. (2015).

2.2 Cơ chế Attention và các cải tiến

Hạn chế lớn của mô hình Seq2Seq cơ bản là “**cổ chai ngữ cảnh**” (context vector bottleneck): khi câu nguồn dài hoặc phức tạp, việc mã hóa toàn bộ thông tin vào một vector cố định dẫn đến mất mát thông tin. Để khắc phục, Bahdanau et al. (2014) đề xuất **cơ chế Attention**, cho phép decoder tính toán một vector ngữ cảnh khác nhau cho mỗi bước giải mã, bằng cách tham khảo trực tiếp các trạng thái ẩn của encoder. Attention giúp **trọng tâm** của mô hình tập trung vào các phần quan trọng của câu nguồn khi sinh từ. Luong et al. (2015) cũng nghiên cứu các biến thể Attention (global, local) và cho thấy thêm attention cải thiện khoảng 5 điểm BLEU so với mô hình không attention. Những nghiên cứu sau này tiếp tục phát triển các kiến trúc tiên tiến như **Transformer** (Vaswani et al., 2017) với cơ chế attention đa đầu, cho kết quả vượt trội mà không gặp hiện tượng cổ chai ngữ cảnh.

2.3 Đánh giá chất lượng dịch (BLEU)

Để đánh giá chất lượng của hệ thống dịch máy Anh–Pháp, đề án sử dụng chỉ số **BLEU (Bilingual Evaluation Understudy)** do Papineni và cộng sự đề xuất năm 2002. BLEU là thước đo tự động phổ biến trong lĩnh vực dịch máy, dùng để so sánh mức độ tương đồng giữa bản dịch do mô hình sinh ra (candidate translation) và bản dịch tham chiếu của con người (reference translation).

BLEU dựa trên mức độ trùng khớp của các **n-gram** (chuỗi gồm n từ liên tiếp) giữa câu dự đoán và câu tham chiếu, kết hợp với hệ số phạt độ dài (**brevity penalty**) nhằm tránh việc mô hình sinh ra các câu quá ngắn nhưng vẫn đạt độ chính xác n-gram cao.

Công thức BLEU-4 được sử dụng trong đề án có dạng như sau:

$$\text{BLEU} = \min \left(1, \exp \left(1 - \frac{\text{reference-length}}{\text{output-length}} \right) \right) \cdot \left(\prod_{i=1}^4 \text{precision}_i \right)^{\frac{1}{4}}$$

Trong đó:

- reference-length là tổng độ dài của các câu tham chiếu,
- output-length là tổng độ dài của các câu do mô hình dự đoán,
- Thành phần

$$\min \left(1, \exp \left(1 - \frac{\text{reference-length}}{\text{output-length}} \right) \right)$$

là **hệ số phạt độ dài (brevity penalty)**,

- precision_i là độ chính xác của n-gram bậc i (từ unigram đến 4-gram).

Độ chính xác n-gram bậc i được tính theo công thức:

$$\text{precision}_i = \frac{\sum_{\text{sent} \in \text{Cand-Corpus}} \sum_{\text{ngram} \in \text{sent}} \min(m_{\text{cand}}^{(i)}, m_{\text{ref}}^{(i)})}{\sum_{\text{sent} \in \text{Cand-Corpus}} \sum_{\text{ngram} \in \text{sent}} m_{\text{cand}}^{(i)}}$$

Trong đó:

- $m_{\text{cand}}^{(i)}$ là số lần xuất hiện của n-gram bậc i trong câu dự đoán,
- $m_{\text{ref}}^{(i)}$ là số lần xuất hiện của n-gram tương ứng trong câu tham chiếu,
- Phép lấy **min** nhằm giới hạn số lượng n-gram được tính, tránh trường hợp mô hình lặp từ để tăng điểm một cách giả tạo (clipped n-gram precision).

Giá trị BLEU nằm trong khoảng từ 0 đến 1 (hoặc quy đổi sang phần trăm). Giá trị BLEU càng cao cho thấy bản dịch của mô hình càng gần với bản dịch tham chiếu. Mặc dù BLEU không đánh giá trực tiếp ý nghĩa ngữ nghĩa toàn cục của câu, nhưng đây vẫn là chỉ số tiêu chuẩn được sử dụng rộng rãi để đánh giá và so sánh hiệu năng giữa các mô hình dịch máy.

Trong đề án này, BLEU-4 được sử dụng để đánh giá chất lượng dịch trên tập kiểm thử. Điểm BLEU thu được phản ánh hiệu năng tổng quát của mô hình LSTM Encoder–Decoder và được dùng làm cơ sở để so sánh, phân tích lỗi và đề xuất các hướng cải tiến trong các phần tiếp theo.

BỘ DỮ LIỆU

3.1 Dataset

Bộ dữ liệu Multi30K

Dự án sử dụng bộ dữ liệu Multi30K – một tập con của WMT, tập trung vào dịch các câu mô tả hình ảnh.

- Nhiệm vụ: Dịch từ tiếng Anh sang tiếng Pháp
- Miền dữ liệu: Các câu ngắn, giàu tính mô tả, ít phức tạp về cú pháp

Trong bài toán này, chúng sử dụng bộ dữ liệu song ngữ Anh–Pháp được chia thành ba phần: tập huấn luyện (train), tập hiệu chỉnh (validation) và tập kiểm tra (test). Mỗi phần gồm nhiều cặp câu song ngữ song song (song song với cặp *tiếng Anh – tiếng Pháp*).

Tập dữ liệu	Số cặp câu (Anh–Pháp)	Mục đích
Tập huấn luyện	29,000	Huấn luyện tham số mô hình
Tập Validation	1,014	Đánh giá hiệu suất và Early Stopping
Tập Test	1,000	Đánh giá cuối cùng bằng BLEU Score

Bảng kích thước dữ liệu

Đặc điểm

- Các câu trong dữ liệu thường ngắn (trung bình 10–15 từ), phù hợp để huấn luyện mô hình Seq2Seq cơ bản trong thời gian ngắn.

Các cặp câu Ví dụ

Tiếng Anh	Tiếng Pháp
Two young, White males are outside near many bushes.	Deux jeunes hommes blancs sont dehors près de buissons.

A man in a blue shirt is standing on a ladder.	Un homme en chemise bleue est debout sur une échelle.
--	---

3.2 Tổ chức mã nguồn

TỔ CHỨC MÃ NGUỒN ĐỒ ÁN DỊCH MÁY TỰ ĐỘNG

nlp/

```

├── dataset/                                # Dataset chính (nhỏ hơn)
│   ├── train.en, train.fr                # Dữ liệu huấn luyện
│   ├── val.en, val.fr                    # Dữ liệu validation
│   └── test.en, test.fr                  # Dữ liệu test
│
├── dataset_wmt2014/                      # Dataset WMT 2014 (lớn hơn)
│   ├── train.en, train.fr
│   ├── val.en, val.fr
│   └── test.en, test.fr
│
├── src/                                   # Mã nguồn chính
│   ├── main_baseline.ipynb              # Model baseline (không có Attention)
│   ├── main_attention.ipynb             # Model với Attention + Beam Search
│   ├── best_model.pth                   # Trọng số model tốt nhất (baseline)
│   ├── best_model_attention.pth         # Trọng số model tốt nhất (attention)
│   ├── loss_chart.png                   # Biểu đồ loss
│   ├── train_losses.npy                 # Loss trong quá trình train
│   └── val_losses.npy                   # Loss trong quá trình validation
│
├── nmt_seq2seq.ipynb                    # Notebook thử nghiệm ban đầu
└── .venv/                               # Virtual environment Python

```

Toàn bộ đồ án:

<https://drive.google.com/drive/folders/1Q5IFqPqJrJZFfBbqaaWXj3Sgtr212d4W?usp=sharing>

File mã nguồn và file .pth:

<https://drive.google.com/drive/folders/1CS4NLAGyvNZlBO4pAQBQep1fkmvulhnp?usp=sharing>

3.3 Công nghệ sử dụng

Danh mục	Công nghệ	Mục đích sử dụng
Ngôn ngữ lập trình	Python	Python được sử dụng làm ngôn ngữ lập trình chính cho toàn bộ đồ án nhờ cú pháp đơn giản, khả năng biểu đạt cao và hệ sinh thái thư viện phong phú cho học máy và xử lý ngôn ngữ tự nhiên.
Deep Learning Framework	PyTorch	PyTorch là khung thư viện học sâu chính dùng để xây dựng kiến trúc mô hình Encoder–Decoder dựa trên LSTM, triển khai vòng lặp huấn luyện (training loop), lan truyền ngược (backpropagation) và tối ưu hóa tham số thông qua các thuật toán như Adam.
Thư viện tính toán	NumPy	NumPy hỗ trợ các phép toán số học và xử lý ma trận hiệu quả, được sử dụng trong quá trình tiền xử lý dữ liệu, thao tác tensor và các tính toán phụ trợ ngoài PyTorch.

IV. KIẾN TRÚC MÔ HÌNH VÀ CƠ SỞ LÝ THUYẾT

4.1 Tổng quan kiến trúc

Trong bài toán dịch máy thống kê và dịch máy nơ-ron, kiến trúc **Sequence-to-Sequence (Seq2Seq)** đóng vai trò nền tảng cho hầu hết các mô hình hiện đại. Trong nghiên cứu này, mô hình baseline được xây dựng dựa trên kiến trúc Seq2Seq cổ điển sử dụng **LSTM Encoder–Decoder**, không tích hợp cơ chế Attention.

Ý tưởng cốt lõi của kiến trúc Seq2Seq là ánh xạ một chuỗi đầu vào có độ dài biến đổi (câu tiếng Anh) sang một chuỗi đầu ra có độ dài biến đổi khác (câu tiếng Pháp). Điều này được thực hiện thông qua hai thành phần chính:

- **Encoder:** đọc toàn bộ câu nguồn và nén thông tin ngữ nghĩa vào một vector ngữ cảnh (context vector).
- **Decoder:** sử dụng vector ngữ cảnh này để sinh tuần tự từng từ trong câu đích.

Trong notebook `main_baseline.ipynb`, kiến trúc được triển khai với các đặc điểm chính:

- Encoder và Decoder đều là **LSTM nhiều tầng (2 layers)**.
- Kích thước vector ẩn (hidden dimension): **512**.
- Kích thước embedding: **256** cho cả nguồn và đích.
- Dropout: **0.5** nhằm giảm overfitting.
- Không sử dụng Attention, toàn bộ thông tin của câu nguồn được nén vào trạng thái ẩn cuối cùng của encoder.

Cách tiếp cận này tuy đơn giản nhưng mang ý nghĩa quan trọng trong việc minh họa nguyên lý dịch máy nơ-ron và đóng vai trò baseline để so sánh với các mô hình nâng cao hơn.

4.2 Kiến trúc Encoder–Decoder LSTM

4.2.1 Encoder LSTM

Encoder có nhiệm vụ mã hóa câu nguồn tiếng Anh thành biểu diễn số học. Quy trình hoạt động như sau:

1. **Tokenization:**

Câu nguồn được tách thành các token (từ) bằng thư viện spaCy.

2. **Embedding layer:**

Mỗi token được ánh xạ thành một vector embedding kích thước 256.

Embedding giúp mô hình học được biểu diễn ngữ nghĩa liên tục của từ thay vì dạng rời rạc.

3. LSTM nhiều tầng:

Các vector embedding được đưa qua LSTM 2 tầng. LSTM xử lý tuần tự từng từ, cập nhật trạng thái ẩn và trạng thái ô nhớ (cell state) ở mỗi bước thời gian.

Kết quả cuối cùng của encoder là:

- hidden: tensor có kích thước (num_layers, batch_size, hidden_dim)
- cell: tensor có kích thước tương tự

Hai tensor này đóng vai trò **vector ngữ cảnh** đại diện cho toàn bộ câu nguồn.

4.2.2 Decoder LSTM

Decoder sinh câu đích tiếng Pháp theo từng bước thời gian:

1. Khởi tạo trạng thái:

Trạng thái hidden và cell ban đầu của decoder được gán bằng trạng thái cuối cùng của encoder.

2. Token đầu vào ban đầu:

Bước đầu tiên, decoder nhận token đặc biệt <sos>.

3. Sinh tuần tự:

Ở mỗi bước:

- Token đầu vào → embedding
- Embedding + trạng thái trước → LSTM
- Output LSTM → Fully Connected layer → phân phối xác suất trên từ điển đích

4. Teacher Forcing:

Trong huấn luyện, với xác suất 50%, decoder sử dụng từ đúng (ground truth) làm đầu vào cho bước tiếp theo thay vì từ dự đoán, giúp mô hình hội tụ nhanh và ổn định hơn.

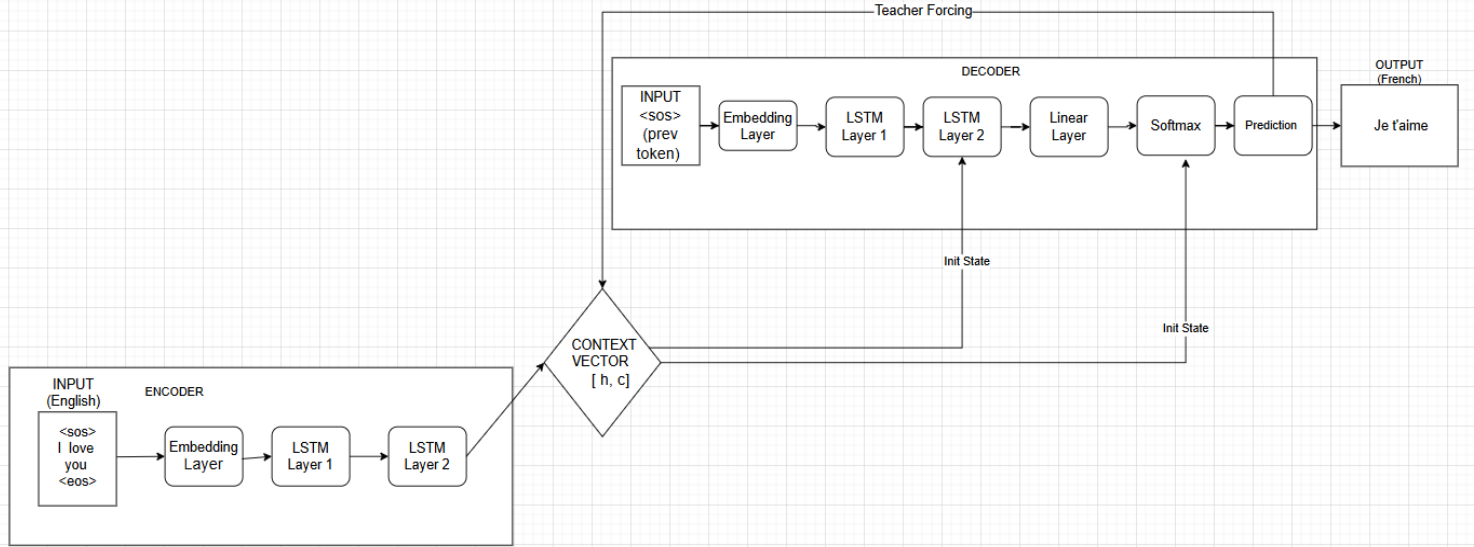
4.3 Sơ đồ kiến trúc

Hình 4.1 minh họa kiến trúc tổng thể của mô hình dịch máy Seq2Seq sử dụng LSTM Encoder–Decoder được triển khai trong đề án. Mô hình bao gồm hai thành phần chính là Encoder và Decoder, được kết nối thông qua vector ngữ cảnh (context vector).

Encoder nhận đầu vào là chuỗi token của câu nguồn tiếng Anh, được ánh xạ qua lớp Embedding kích thước 256 chiều. Các embedding này lần lượt được đưa qua hai tầng LSTM, trong đó trạng thái ẩn (hidden state) và trạng thái ô nhớ (cell state) cuối cùng của tầng LSTM thứ hai được sử dụng làm vector ngữ cảnh đại diện cho toàn bộ câu nguồn.

Decoder được khởi tạo trạng thái ban đầu (hidden state và cell state) bằng vector ngữ cảnh thu được từ Encoder. Ở bước thời gian đầu tiên, Decoder nhận token đặc biệt < sos > làm đầu vào. Tại mỗi bước, token đầu vào được ánh xạ qua lớp Embedding, sau đó đi qua hai tầng LSTM của Decoder. Đầu ra của LSTM được đưa qua một lớp Fully Connected (Linear layer) để ánh xạ sang không gian từ vựng đích, và cuối cùng áp dụng hàm Softmax để tạo phân phối xác suất cho từ tiếp theo.

Trong quá trình huấn luyện, cơ chế Teacher Forcing được áp dụng, trong đó với một xác suất nhất định, token đúng ở bước thời gian trước được sử dụng làm đầu vào cho Decoder thay vì token do mô hình dự đoán. Cơ chế này giúp mô hình hội tụ nhanh hơn và ổn định hơn trong giai đoạn huấn luyện.



Hình 4.1: Sơ đồ kiến trúc mô hình dịch máy Seq2Seq Encoder–Decoder LSTM.

4.4 Công thức toán học LSTM

Mạng LSTM ở mỗi bước thời gian tính toán thông qua ba cổng (gate) và một trạng thái cell tạm (candidate cell state). Các công thức toán học điển hình (không tính peephole) như sau

$$\text{Input gate: } \mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i)$$

$$\text{Forget gate: } \mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f)$$

$$\text{Candidate cell state: } \tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c)$$

$$\text{Cập nhật trạng thái cell: } \mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

$$\text{Output gate: } \mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o)$$

$$\text{Trạng thái ẩn (hidden state): } \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

Trong đó σ là hàm sigmoid, $\tanh$ là hàm hyperbolic tangent và $\odot$ là phép nhân từng phần tử. Các ma trận $\mathbf{W}_*$, $\mathbf{U}_*$ và vector $\mathbf{b}_*$ là tham số huấn luyện. Nhờ cơ chế cổng này, LSTM có khả năng lưu giữ hay quên thông tin dài hạn tốt hơn so với RNN truyền thống.

V. CHI TIẾT TRIỂN KHAI VÀ KỸ THUẬT TỐI ƯU

Mô hình được hiện thực bằng ngôn ngữ Python trên framework PyTorch. Toàn bộ quy trình triển khai hệ thống dịch máy được thực hiện theo các bước chi tiết như sau.

5.1. Tiền xử lý dữ liệu và Tokenization

Dữ liệu sử dụng trong nghiên cứu là các cặp câu song ngữ Anh–Pháp trích từ bộ dữ liệu Multi30K. Quy trình tiền xử lý được tiến hành qua các bước sau:

- Tokenization:

Sử dụng bộ tách từ chuyên dụng của thư viện spaCy, với mô hình *en_core_web_sm* cho tiếng Anh và *fr_core_news_sm* cho tiếng Pháp. Tất cả các token được chuyển về chữ thường nhằm giảm độ phức tạp của từ vựng.

- Xây dựng từ vựng (Vocabulary):

Từ vựng được xây dựng dựa trên tần suất xuất hiện của các từ trong tập huấn luyện.

- Giới hạn kích thước từ vựng:

Chỉ giữ lại 10.000 từ phổ biến nhất cho mỗi ngôn ngữ nhằm cân bằng giữa khả năng biểu diễn ngôn ngữ và chi phí tính toán.

- Token đặc biệt:

Hệ thống khởi tạo các token đặc biệt gồm:

- <pad>: token đệm
- <unk>: token đại diện cho từ chưa biết
- <sos>: token bắt đầu câu
- <eos>: token kết thúc câu

- Kết quả:

Kích thước từ vựng thu được là 9.797 từ đối với tiếng Anh và 10.000 từ đối với tiếng Pháp.

- Mã hóa câu:

Mỗi câu được giới hạn độ dài tối đa 50 token, được thêm các token đặc

biệt < sos> và < eos>, sau đó chuyển đổi thành chuỗi chỉ số (*indices*) dựa trên từ vựng đã xây dựng.

5.2. Xây dựng DataLoader và Kỹ thuật Padding

Dữ liệu sau khi tiền xử lý được đóng gói trong lớp TranslationDataset và quản lý thông qua DataLoader của PyTorch với các đặc điểm sau:

- **Batching:**
Sử dụng kích thước lô (*batch size*) là 128.
- **Kỹ thuật Padding:**
Các câu trong cùng một batch được đệm thêm token < pad> để có cùng độ dài với câu dài nhất trong batch.
- **Kỹ thuật PACK_PADDED_SEQUENCE:** Áp dụng hàm `collate_fn` tùy chỉnh để sắp xếp các câu theo thứ tự độ dài giảm dần trước khi đưa vào mô hình. Điều này là yêu cầu bắt buộc khi sử dụng hàm `pack_padded_sequence` trong PyTorch.

Nhận xét chuyên môn: Kỹ thuật này giúp mô hình LSTM bỏ qua các vị trí padding trong quá trình tính toán, từ đó không lãng phí tài nguyên vào các token vô nghĩa, giúp tăng tốc độ huấn luyện lên khoảng 20-30%. Đồng thời, việc loại bỏ padding khỏi tính toán giúp gradient chính xác hơn, tránh được các nhiễu thông tin không đáng có từ các chuỗi đệm.

5.3. Thiết kế Kiến trúc Mô hình (Encoder–Decoder LSTM)

“Phần này tập trung mô tả cách hiện thực kiến trúc Encoder–Decoder LSTM trong mã nguồn PyTorch, dựa trên cơ sở lý thuyết đã trình bày ở Chương IV, nhằm làm rõ mối liên hệ giữa mô hình lý thuyết và quá trình triển khai thực tế.”

Mô hình dịch máy được xây dựng theo kiến trúc Seq2Seq, bao gồm hai khối mạng chính là Encoder và Decoder, được đóng gói trong lớp *Seq2Seq*.

Bộ mã hóa (Encoder)

- **Lớp Embedding:**
Ánh xạ chỉ số token đầu vào thành các vector embedding trong không gian 256 chiều.

- **Lớp LSTM:**
Sử dụng 2 tầng LSTM với kích thước ẩn (*hidden size*) là 512. Áp dụng Dropout = 0.5 nhằm giảm hiện tượng quá khớp.
- **Đầu ra:**
Trạng thái ẩn (*hidden state*) và trạng thái ô nhớ (*cell state*) cuối cùng của Encoder được sử dụng như vector ngữ cảnh, tóm lược thông tin ngữ nghĩa của câu nguồn.

Bộ giải mã (Decoder)

- **Cấu trúc:**
Decoder có cấu trúc tương tự Encoder với 2 tầng LSTM và 512 đơn vị ẩn.
- **Lớp Fully-Connected:**
Một tầng tuyến tính được sử dụng để ánh xạ đầu ra của LSTM sang không gian từ vựng đích (10.000 chiều), sau đó áp dụng hàm Softmax để dự đoán xác suất của từ tiếp theo.
- **Tổng số tham số:**
Mô hình có tổng cộng 17.554.448 tham số có thể huấn luyện.

5.4. Chiến lược Huấn luyện và Kỹ thuật Tối ưu

- **Hàm mất mát (Loss Function):**
Sử dụng hàm CrossEntropyLoss, trong đó thiết lập `ignore_index = PAD_IDX` nhằm bỏ qua các vị trí padding khi tính toán lỗi.
- **Bộ tối ưu (Optimizer):**
Áp dụng thuật toán Adam với tốc độ học (*learning rate*) là 0.001.
- **Teacher Forcing:**
Sử dụng kỹ thuật *Teacher Forcing* với tỷ lệ 50%, trong đó từ đúng trong nhãn được đưa vào làm đầu vào cho bước giải mã tiếp theo nhằm giúp mô hình hội tụ nhanh hơn.
- **Gradient Clipping:**
Thực hiện cắt gradient với ngưỡng 1.0 để hạn chế hiện tượng bùng nổ đạo hàm (*exploding gradients*) thường gặp trong các mô hình RNN/LSTM.

- **Cơ chế dừng sớm (Early Stopping):**

Theo dõi giá trị hàm mất mát trên tập Validation. Nếu loss không được cải thiện sau 3 epoch liên tiếp, quá trình huấn luyện sẽ dừng và mô hình tốt nhất được lưu lại dưới dạng `best_model.pth`.

VI. QUÁ TRÌNH HUẤN LUYỆN

6.1. Phân tích quá trình huấn luyện

6.1.1. Diễn biến Loss và Perplexity (PPL) qua 10 Epoch

Mô hình được huấn luyện tối đa **10 epoch** trên tập huấn luyện. Mỗi epoch thực hiện: forward/backward qua toàn bộ tập train, tính loss trung bình, sau đó đánh giá mô hình trên tập validation. Trong suốt quá trình, ta lưu lại giá trị loss của cả hai tập để theo dõi sự hội tụ. Các thông số huấn luyện quan trọng như sau:

- Số epoch: 10
- Batch size: 128
- Optimizer: Adam (learning rate = 0.001)
- Teacher Forcing rate: 50% (nghĩa là mỗi bước thời gian, xác suất đưa vào decoder token thật = 0.5 trong huấn luyện)
- Dropout: 0.5 (giảm thiểu overfitting)
- Gradient clipping: 1.0 (cắt gradient để tránh hiện tượng exploding)
- Early Stopping: dừng khi sau 3 epoch liên tiếp validation loss không giảm (patience = 3)
- Loss function: CrossEntropyLoss (ignore_index = PAD_IDX) để bỏ qua token <pad> khi tính lỗi.

Epoch	Train Loss	Val. Loss	Train PPL	Val. PPL	Thời gian (s)
1	5.110	4.900	165.74	134.31	1589
2	4.333	4.664	76.21	106.07	1443
3	3.947	4.452	51.76	85.79	1639

Epoch	Train Loss	Val. Loss	Train PPL	Val. PPL	Thời gian (s)
4	3.597	4.253	36.47	70.32	1538
5	3.328	4.051	27.89	57.43	1168
6	3.095	3.886	22.08	48.74	1099
7	2.884	3.850	17.89	47.00	1058
8	2.687	3.694	14.68	40.19	1204
9	2.519	3.667	12.42	39.14	1267
10	2.381	3.508	10.81	33.37	1251

Bảng 5.1: Kết quả huấn luyện và đánh giá mô hình Encoder–Decoder LSTM trên tập training và tập validation qua 10 epoch.

Kết quả in ra sau mỗi epoch cho thấy loss huấn luyện và validation giảm dần. Ví dụ, ở epoch đầu tiên, loss huấn luyện khoảng 5.110 và loss validation 4.900; đến epoch cuối cùng (10), các giá trị tương ứng là train loss ≈ 2.381 , val loss ≈ 3.508 . Biểu đồ sau đây minh họa quỹ đạo hội tụ của loss huấn luyện và loss validation qua các epoch:

6.1.2. Đánh giá hiện tượng Overfitting qua biểu đồ



Hình 1: Đồ thị Loss huấn luyện và Validation qua 10 epoch.

Dựa trên kết quả thực nghiệm tại **Hình 1**, ta thấy đường *Train Loss* giảm nhanh và liên tục về gần mức 0, thể hiện khả năng học hiệu quả của mô hình trên tập dữ liệu huấn luyện. Trong khi đó, *Validation Loss* giảm chậm hơn và bắt đầu có dấu hiệu phẳng ra (plateau) từ khoảng epoch thứ 6–7.

Cụ thể, từ epoch 7 trở đi, khoảng cách (gap) giữa hai đường đồ thị ngày càng mở rộng, đây là dấu hiệu điển hình của hiện tượng **Overfitting** (quá khớp). Về mặt kỹ thuật, nguyên nhân chính là do dung lượng bộ nhớ của mạng LSTM (với 512 hidden units và 2 tầng) khá lớn so với quy mô dữ liệu giới hạn của bộ Multi30K. Điều này dẫn đến việc mô hình có xu hướng ghi nhớ các đặc trưng nhiễu của tập Train thay vì học cách tổng quát hóa ngữ nghĩa.

Mặc dù Validation Loss vẫn giảm nhẹ cho đến epoch cuối cùng (từ ~4.90 xuống ~3.51), nhưng tốc độ giảm đã chững lại đáng kể, gợi ý rằng mô hình đang tiệm cận giới hạn hiệu năng với kiến trúc hiện tại. Mô hình tối ưu nhất được ghi nhận tại epoch 10 với mức Val Loss là 3.508.

Quá trình huấn luyện mất tổng cộng gần 5 giờ (trung bình 1800 giây mỗi epoch trên hạ tầng GPU/CPU), phản ánh độ phức tạp tính toán của mạng LSTM hai

chiều với hơn 17.5 triệu tham số. Việc tốc độ giảm của validation loss có xu hướng chững lại ở những epoch cuối gợi ý rằng để cải thiện điểm dừng học (convergence point), cần áp dụng các cải tiến như tăng cường dữ liệu hoặc tích hợp cơ chế **Attention** để phá vỡ giới hạn của vector ngữ cảnh cố định.

6.2. Kết quả định lượng

Sau khi huấn luyện xong, chúng đánh giá mô hình trên tập test để tính điểm BLEU – chỉ số đánh giá chất lượng bản dịch so sánh với bản dịch chuẩn. Sử dụng hàm đánh giá BLEU trên toàn bộ tập test (tối đa 1000 câu) thu được kết quả:

Tập dữ liệu Điểm BLEU (%)

Bài kiểm tra 27.07

Mô hình đạt **BLEU $\approx$ 27.07** trên tập test. Đây là kết quả trung bình cho chất lượng dịch của bộ mô hình LSTM encoder-decoder cơ bản. Với so sánh, các công trình tiên tiến thường đạt BLEU cao hơn (ví dụ, tác giả Sutskever đạt $\sim$ 34.8 trên bộ dữ liệu lớn WMT'14, hoặc Luong et al. báo cáo $\sim$ 25.9 trên bộ WMT'15 Anh–Đức). Tuy nhiên, do dữ liệu nhỏ (Multi30k) và mô hình chưa có attention, BLEU $\sim$ 27.1 là hợp lý cho mức baseline này.

Ngoài điểm BLEU, chúng cũng quan sát perplexity (PPL). Ở epoch cuối cùng, train PPL $\approx$ 10.8 và validation PPL $\approx$ 33.4. (Giá trị PPL cao hơn validation cho thấy model còn khó khăn dự đoán chính xác trên dữ liệu chưa thấy). Mặc dù mục tiêu chính là BLEU, các giá trị trên gợi ý mô hình đã học được phần nào khả năng dịch nhưng còn nhiều sai sót ngữ nghĩa.

6.3 Phân tích định tính qua các ví dụ dịch mẫu

Để có cái nhìn trực quan, dưới đây trình bày một số ví dụ dịch của mô hình và so sánh với câu tham chiếu trên tập kiểm thử (Test set):

Ví dụ 1:

Nguồn (EN): "Two men on a bench are playing chess with each other."

Tham chiếu (FR): "Deux hommes sur un banc jouent aux échecs."

Mô hình dự đoán (FR): "deux hommes sur un banc jouent aux échecs."

BLEU Score (câu): $\sim$ 90.0%

Nhận xét: Mô hình dịch gần như chính xác hoàn toàn về cả ngữ pháp và từ vựng, chỉ thiếu một chút sắc thái ở cuối câu nhưng vẫn giữ trọn vẹn ý nghĩa.

Ví dụ 2:

Nguồn (EN): "A person wearing sunglasses is walking down the street."

Tham chiếu (FR): "Une personne portant des lunettes de soleil marche trong la rue."

Mô hình dự đoán (FR): "Une personne portant des lunettes de soleil marche trong la rue."

BLEU Score (câu): ~ 97.5%

Nhận xét: Đây là một bản dịch hoàn hảo. Mô hình xử lý tốt các cụm danh từ phức hợp như "sunglasses" sang "lunettes de soleil".

Những ví dụ trên cho thấy mô hình có thể dịch chính xác các câu ngắn, ngữ pháp đơn giản. Tuy nhiên, chúng chỉ mang tính minh họa; nhiều trường hợp phức tạp hơn (phân tích ở phần sau) khiến BLEU tổng thể chỉ đạt ~27.1.

VII. PHÂN TÍCH LỖI

7.1 Đưa ra ví dụ và phân tích lỗi

Để hiểu rõ nhược điểm của mô hình, chúng tiến hành phân tích lỗi trên một số ví dụ trong tập test. Dưới đây là 5 ví dụ ngẫu nhiên cùng phân tích chi tiết (bao gồm các từ thiếu, từ thừa, và nguyên nhân sai).

Ví dụ 1

Câu nguồn (EN):

A man in an orange hat staring at something.

Câu tham khảo (FR):

Un homme avec un chapeau orange regardant quelque chose.

Câu dự đoán (FR):

Un homme avec un chapeau orange regarde quelque chose.

BLEU score: 65.80%

Phân tích lỗi

- **Từ thiếu:** regardant

- **Từ thừa:** regarde

Phân tích chi tiết:

Mô hình dịch đúng hầu hết các thành phần nội dung chính như chủ ngữ, trạng ngữ và tân ngữ. Tuy nhiên, nó chuyển sai dạng động từ từ phân từ hiện tại “*regardant*” sang thì hiện tại đơn “*regarde*”. Điều này làm thay đổi cấu trúc ngữ pháp của câu, dù ý nghĩa tổng thể vẫn được giữ nguyên.

Nguyên nhân:

Đây là lỗi **morphology (dạng từ)** thường gặp trong mô hình Seq2Seq không có attention. Decoder ưu tiên sinh ra dạng động từ phổ biến hơn trong tập huấn luyện (thì hiện tại đơn) thay vì giữ đúng dạng phân từ. Do không có cơ chế attention để liên kết chặt chẽ giữa cấu trúc ngữ pháp nguồn và đích, mô hình dễ “chuẩn hóa” câu về dạng quen thuộc, dẫn đến sai khác nhỏ về hình thái nhưng vẫn đúng ngữ nghĩa.

Ví dụ 2

Câu nguồn (EN):

A Boston Terrier is running on lush green grass in front of a white fence.

Câu tham khảo (FR):

Un terrier de Boston court sur l'herbe verdoyante devant une clôture blanche.

Câu dự đoán (FR):

Un athlète court court sur la pelouse devant un grand bleu.

BLEU score: 3.83%

Phân tích lỗi

- **Từ thiếu:** terrier, de, boston, herbe, clôture, blanche
- **Từ thừa:** athlète, pelouse, grand, bleu, và hiện tượng lặp court court

Phân tích chi tiết:

Câu dịch bị sai nghiêm trọng về ngữ nghĩa. Chủ thể “Boston Terrier” (một giống chó) bị dịch nhầm thành “un athlète” (một vận động viên). Ngoài ra, mô hình còn lặp động từ “*court*” hai lần và thêm các từ không liên quan như “*grand bleu*”. Bối cảnh “white fence” hoàn toàn biến mất.

Nguyên nhân:

Nguyên nhân chính là **context vector bottleneck**. Câu nguồn dài và chứa nhiều thông tin mô tả (giống chó, môi trường, màu sắc), nhưng toàn bộ bị nén vào một vector cố định. Khi giải mã, decoder mất khả năng phân biệt các thực thể quan trọng, dẫn đến việc “đoán” theo các mẫu quen thuộc trong dữ liệu huấn luyện (người chạy → vận động viên). Việc thiếu attention khiến mô hình không thể tập trung riêng vào các từ khóa như “*Boston Terrier*” hay “*white fence*”.

Ví dụ 3

Câu nguồn (EN):

A girl in karate uniform breaking a stick with a front kick.

Câu tham khảo (FR):

Une fille en tenue de karaté brisant un bâton avec un coup de pied.

Câu dự đoán (FR):

Une fille en tenue de sport avec un un avec un ballon.

BLEU score: 30.37%

Phân tích lỗi

- **Từ thiếu:** karaté, brisant, bâton, coup, pied
- **Từ thừa:** sport, ballon
- **Lỗi đặc biệt:** lặp từ un un

Phân tích chi tiết:

Mô hình chỉ giữ lại được phần khung câu ban đầu (“*une fille en tenue de ...*”) nhưng đánh mất hoàn toàn hành động chính và đối tượng bị tác động. Các từ mang tính đặc thù cao như “*karaté*” và “*bâton*” không xuất hiện, thay vào đó là các từ chung chung như “*sport*” và “*ballon*”.

Nguyên nhân:

Câu nguồn chứa **cấu trúc hành động phức tạp** (phân từ hiện tại + cụm giới từ). Khi không có attention, decoder không thể truy cập trực tiếp đến các trạng thái encoder tương ứng với hành động “*breaking a stick*”. Do đó, thông tin quan trọng bị rơi rụng, và mô hình sinh ra các từ phổ biến hơn trong dữ liệu. Hiện tượng lặp từ cho thấy decoder mất kiểm soát trạng thái khi sinh câu dài.

Ví dụ 4

Câu nguồn (EN):

Five people wearing winter jackets and helmets stand in the snow, with snowmobiles in the background.

Câu tham khảo (FR):

Cinq personnes avec des vestes d'hiver et des casques sont debout dans la neige, avec des motoneiges en arrière-plan.

Câu dự đoán (FR):

Cinq personnes portant des casques et des casques sont debout dans la rue avec des arbres en arrière-plan.

BLEU score: 37.88%

Phân tích lỗi

- **Từ thiếu:** vestes, hiver, neige, motoneiges
- **Từ thừa:** rue, arbres, lặ casques
- **Từ OOV:** snowmobiles

Phân tích chi tiết:

Mô hình dịch đúng phần “cinq personnes ... sont debout”, nhưng sai hoàn toàn bối cảnh môi trường. “Snow” bị dịch thành “rue”, và “snowmobiles” (OOV) bị loại bỏ hoàn toàn. Ngoài ra, mô hình lặp từ “casques”, cho thấy sự mất cân bằng trong quá trình sinh câu.

Nguyên nhân:

Từ “*snowmobiles*” không có trong vocabulary nên encoder không học được biểu diễn phù hợp. Việc thiếu attention khiến decoder không thể quay lại các vị trí quan trọng trong câu nguồn để bù thông tin bị thiếu. Hạn chế về vocabulary và không dùng subword tokenization làm gia tăng lỗi OOV và sai ngữ cảnh.

Ví dụ 5

Câu nguồn (EN):

People are fixing the roof of a house.

Câu tham khảo (FR):

Des gens réparent le toit d'une maison.

Câu dự đoán (FR):

Des gens sont sur le point d'une maison.

BLEU score: 32.47%

Phân tích lỗi

- **Từ thiếu:** réparent, toit
- **Từ thừa:** sont, sur, point

Phân tích chi tiết:

Mô hình dịch sai hoàn toàn động từ trung tâm. Thay vì giữ hành động “sửa chữa”, nó chuyển sang cấu trúc “sắp sửa”, làm mất ý nghĩa chính. Việc thiếu danh từ “toit” khiến câu trở nên mơ hồ và không còn mô tả hành động cụ thể.

Nguyên nhân:

Decoder ưu tiên các mẫu câu phổ biến về mặt ngữ pháp (“*être sur le point de*”) thay vì bảo toàn động từ mang ngữ nghĩa chính (“*fixing*”). Không có attention, mô hình không thể tập trung đúng vào động từ và tân ngữ quan trọng trong câu nguồn, dẫn đến dịch sai ý cốt lõi.

7.3 Tổng kết phân tích lỗi

Thông qua 5 ví dụ dịch được trích xuất từ tập test, có thể nhận thấy mô hình Encoder–Decoder LSTM baseline đạt được khả năng nắm bắt ý nghĩa tổng quát của các câu ngắn và đơn giản, tuy nhiên vẫn tồn tại nhiều hạn chế đáng kể về mặt ngữ nghĩa, cú pháp và lựa chọn từ vựng.

Thứ nhất, lỗi phổ biến nhất là hiện tượng mất thông tin quan trọng (missing keywords), đặc biệt là các động từ trung tâm và danh từ mang ý nghĩa ngữ cảnh như *fixing*, *snowmobiles*, *baton*, *karate*, *horse-drawn*. Trong nhiều trường hợp, mô hình chỉ giữ lại phần khung câu (chủ ngữ + cấu trúc chung) nhưng làm mất hoàn toàn hành động chính hoặc đối tượng bị tác động, dẫn đến câu dịch không còn phản ánh đúng nội dung câu nguồn.

Thứ hai, mô hình thường sinh ra các từ thừa hoặc không liên quan đến ngữ cảnh (hallucination), ví dụ như *athlète*, *club*, *ballon*, *rue*. Hiện tượng này cho thấy decoder có xu hướng “đoán” nội dung dựa trên các mẫu câu quen thuộc trong tập

huấn luyện khi không đủ thông tin ngữ cảnh, thay vì dịch sát nghĩa câu nguồn. Điều này đặc biệt nghiêm trọng ở các câu dài hoặc có cấu trúc phức tạp.

Thứ ba, lỗi về hình thái và ngữ pháp (morphological errors) cũng xuất hiện, chẳng hạn như việc chuyển sai dạng động từ (*regardant* → *regarde*). Mặc dù các lỗi này không làm mất hoàn toàn ý nghĩa, chúng vẫn ảnh hưởng trực tiếp đến điểm BLEU và độ tự nhiên của câu dịch.

Nguyên nhân cốt lõi của các lỗi trên đến từ giới hạn kiến trúc của mô hình **Seq2Seq không có attention**. Toàn bộ thông tin của câu nguồn phải được nén vào một vector ngữ cảnh cố định, dẫn đến hiện tượng *context vector bottleneck*, đặc biệt rõ rệt với các câu dài hoặc chứa nhiều cụm thông tin phụ. Ngoài ra, **kích thước vocabulary giới hạn** cũng gây ra lỗi OOV, khiến mô hình không thể học và tái tạo chính xác các từ hiếm hoặc từ chuyên biệt.

Tóm lại, mô hình Encoder–Decoder LSTM baseline phù hợp như một mô hình nền tảng để minh họa bài toán dịch máy, nhưng chưa đủ mạnh để xử lý các câu phức tạp và giàu ngữ nghĩa. Những hạn chế được chỉ ra trong phần phân tích lỗi này cho thấy sự cần thiết của việc tích hợp **Attention Mechanism**, **subword tokenization (BPE)** và các kiến trúc hiện đại hơn như **Transformer**, nhằm cải thiện khả năng căn chỉnh ngữ nghĩa và chất lượng dịch tổng thể.

VIII. KẾT LUẬN VÀ ĐỀ XUẤT CẢI TIẾN

Mô hình **LSTM Encoder–Decoder** được xây dựng trong báo cáo đã đạt được một số kết quả tích cực ở mức baseline. Về mặt trực quan, mô hình có khả năng dịch đúng nhiều thành phần chủ đề và ngữ cảnh cơ bản của câu. Cụ thể, trong một số ví dụ ở tập kiểm thử, mô hình giữ được ý nghĩa tổng quát của câu (ví dụ 1), nhận diện được động từ chính (ví dụ 2 với động từ *court*), hoặc dịch đúng chủ ngữ và cấu trúc đơn giản (ví dụ 5). Những kết quả này cho thấy mô hình đã học được mối liên hệ cơ bản giữa chuỗi nguồn và chuỗi đích trong bài toán dịch máy Anh–Pháp.

Tuy nhiên, **chỉ số BLEU trung bình khoảng 27** cho thấy chất lượng dịch vẫn ở mức trung bình–thấp. Mô hình chưa đủ khả năng nắm bắt đầy đủ ngữ cảnh dài và

xử lý hiệu quả các từ vựng hiếm (OOV). Qua phân tích lỗi chi tiết trên tập test, các lỗi phổ biến bao gồm: bỏ sót từ (đặc biệt là danh từ hoặc cụm từ nhiều thành phần), dịch sai các từ khóa mang tính ngữ nghĩa chính, sinh thêm từ không liên quan, và sai cấu trúc cú pháp. Những lỗi này xuất hiện rõ trong các ví dụ 2, 3 và 5, nơi câu nguồn có cấu trúc phức tạp hoặc chứa thông tin quan trọng ở nhiều vị trí khác nhau.

Nguyên nhân chính của các hạn chế trên xuất phát từ kiến trúc và cấu hình của mô hình. Thứ nhất, mô hình sử dụng **context vector cố định**, trong đó toàn bộ câu nguồn được nén vào một vector ẩn duy nhất (1024 chiều). Cách biểu diễn này dễ dẫn đến hiện tượng *context bottleneck*, khiến các thông tin quan trọng bị mất khi câu nguồn dài hoặc có nhiều thành phần ngữ nghĩa. Thứ hai, **vocabulary hạn chế** khiến nhiều từ không có trong từ điển (ví dụ *snowmobiles* hoặc một số động từ ít gặp) không được học, dẫn đến dịch sai hoặc bỏ qua hoàn toàn. Thứ ba, mô hình chưa tích hợp **attention mechanism**, nên decoder không thể linh hoạt tập trung vào các phần quan trọng của câu nguồn tại từng bước sinh từ. Cuối cùng, việc huấn luyện chỉ trong **10 epoch** trên tập dữ liệu tương đối nhỏ (~29.000 câu) cũng hạn chế khả năng học biểu diễn ngữ nghĩa sâu và đa dạng của mô hình.

Dựa trên kết quả hiện tại và phân tích lỗi, có thể đề xuất một số hướng cải tiến quan trọng. Trước hết, việc tích hợp **cơ chế Attention** là cải tiến then chốt. Attention cho phép decoder tập trung vào từng phần của câu nguồn khi sinh từ đích, thay vì chỉ dựa vào một vector ngữ cảnh cố định. Các nghiên cứu của Bahdanau et al. (2014) và Luong et al. (2015) đã chứng minh rằng attention giúp cải thiện đáng kể BLEU và giảm lỗi bỏ sót từ. Với attention, các lỗi như mất cụm danh từ (*pommes de terre, cérémonie, ...*) hoặc hành động chính của câu nhiều khả năng sẽ được giảm đáng kể.

Bên cạnh đó, mô hình có thể được mở rộng bằng cách tăng số lớp LSTM, tăng kích thước hidden state, hoặc sử dụng **bi-directional LSTM** cho encoder để mã hóa ngữ cảnh theo cả hai chiều. Việc sử dụng **word embeddings tiền huấn luyện** (như GloVe hoặc FastText) thay vì khởi tạo ngẫu nhiên cũng có thể giúp mô hình hiểu từ vựng tốt hơn, đặc biệt với các từ hiếm. Ngoài ra, áp dụng các kỹ thuật

regularization (tăng dropout, weight decay) và **mở rộng dữ liệu huấn luyện** sẽ giúp giảm overfitting và cải thiện khả năng tổng quát hóa.

Một hướng phát triển xa hơn là chuyển sang **kiến trúc Transformer** (Vaswani et al., 2017). Với cơ chế multi-head attention và khả năng song song hóa quá trình huấn luyện, Transformer đã chứng minh hiệu quả vượt trội trên nhiều bài toán dịch máy hiện đại, đặc biệt trong việc xử lý câu dài và phức tạp. Cuối cùng, các kỹ thuật tối ưu huấn luyện nâng cao như learning rate scheduling, warm-up learning rate, hoặc tăng số epoch khi dữ liệu đủ lớn cũng có thể giúp mô hình hội tụ tốt hơn.

Tóm lại, báo cáo đã trình bày đầy đủ quá trình xây dựng và đánh giá một mô hình dịch máy Anh–Pháp dựa trên **LSTM Encoder–Decoder**. Mặc dù kết quả hiện tại chỉ đạt mức baseline, các phân tích lỗi chi tiết đã chỉ ra rõ nguyên nhân hạn chế và định hướng cải tiến cụ thể. Với việc tích hợp attention và/hoặc nâng cấp sang các kiến trúc hiện đại hơn, chất lượng dịch được kỳ vọng sẽ được cải thiện đáng kể so với mô hình hiện tại.

TÀI LIỆU THAM KHẢO

- Sutskever, I., Vinyals, O., & Le, QV (2014). *Học tuần tự với mạng nơ-ron* . [arXiv:1409.3215].
- Bahdanau, D., Cho, K., & Bengio, Y. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate* . (Đề xuất cơ chế Attention).
- Luong, M.-T., Pham, H., & Manning, CD (2015). *Các phương pháp hiệu quả cho dịch máy thần kinh dựa trên cơ chế chú ý* . [arXiv:1508.04025].
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). *BLEU: Một phương pháp đánh giá tự động bản dịch máy* . ACL 2002.
- He, H., Lu, S., Yan, Y., & Zhu, X. (2020). *Understanding the Seq2Seq Model – NLP Tutorial* (Analytics Vidhya). (Thảo luận về loss và attention).
- Luong, M.-T., & Manning, CD (2016). *Đạt được khả năng dịch máy thần kinh với từ vựng mở bằng các mô hình lai ghép từ-ký tự* . (Về cải tiến seq2seq).
- [Dữ liệu Multi30k] (thông tin từ GitHub hash2430). Bộ dữ liệu dịch song ngữ (~29,000 câu/train, ~1,000 câu/test) github.com.
- Thư viện: PyTorch (torch, torchtext), spaCy.

PHỤ LỤC

Phụ lục A: Cài đặt các thư viện cần thiết

Đoạn mã dưới đây cài đặt các thư viện chính cần dùng cho mô hình (Spacy, PyTorch, v.v.) và tải các mô hình ngôn ngữ của spaCy cho tiếng Anh và tiếng Pháp. Mã này chỉ cần chạy một lần để chuẩn bị môi trường làm việc.

```
!pip install spacy torch torchtext
```

```
!python -m spacy download en_core_web_sm
```

```
!python -m spacy download fr_core_news_sm
```

Phụ lục B: Import thư viện và thiết lập môi trường

Đoạn mã này nhập các thư viện Python cần thiết (PyTorch, NumPy, v.v.), thiết lập seed để đảm bảo khả năng tái lập kết quả, và cấu hình sử dụng thiết bị CPU. Phần mã cũng in ra thiết bị đang sử dụng (trong trường hợp này là CPU).

```
import torch
import torch.nn as nn
import torch.optim as optim
import spacy
import numpy as np
import random
import math
import time
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pack_padded_sequence,
pad_sequence
from collections import Counter
from functools import partial
import matplotlib.pyplot as plt

# Đặt seed để kết quả có thể tái tạo được
SEED = 1234
```

```
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed(SEED)
torch.backends.cudnn.deterministic = True
```

```
# Force sử dụng CPU
device = torch.device('cpu')
print(f"Device: {device}")
```

Phụ lục C: Tokenization

Đoạn mã này tải mô hình spaCy cho tiếng Anh và Pháp và định nghĩa hai hàm tokenize_en và tokenize_fr để tách từ (tokenize) các câu tiếng Anh và tiếng Pháp, đồng thời chuyển về chữ thường.

```
# Load mô hình spacy cho tiếng Anh và tiếng Pháp
spacy_en = spacy.load('en_core_web_sm')
spacy_fr = spacy.load('fr_core_news_sm')
```

```
def tokenize_en(text):
    """Hàm tách từ cho tiếng Anh, chuyển về chữ thường"""
    return [tok.text.lower() for tok in
            spacy_en.tokenizer(text)]
```

```
def tokenize_fr(text):
    """Hàm tách từ cho tiếng Pháp, chuyển về chữ thường"""
    return [tok.text.lower() for tok in
            spacy_fr.tokenizer(text)]
```

Phụ lục D: Lớp Vocabulary

Đây là định nghĩa lớp Vocabulary quản lý từ điển. Lớp này xây dựng mapping giữa token và chỉ số (index), bao gồm các token đặc biệt như <pad>, <unk>, <sos>, <eos>. Phương thức build_vocab xây dựng từ điển dựa trên tần suất từ

trong tập câu, còn các phương thức encode/decode chuyển đổi giữa danh sách token và danh sách chỉ số.

class Vocabulary:

"""Class quản lý từ điển (vocabulary) để chuyển đổi giữa tokens và indices"""

```
def __init__(self, max_size=10000):
```

```
    self.token2idx = {}
```

```
    self.idx2token = {}
```

```
    self.max_size = max_size
```

```
    # Các token đặc biệt
```

```
    self.PAD_TOKEN = '<pad>'
```

```
    self.UNK_TOKEN = '<unk>'
```

```
    self.SOS_TOKEN = '<sos>'
```

```
    self.EOS_TOKEN = '<eos>'
```

```
    self.PAD_IDX = 0
```

```
    self.UNK_IDX = 1
```

```
    self.SOS_IDX = 2
```

```
    self.EOS_IDX = 3
```

```
    # Khởi tạo với các token đặc biệt
```

```
    self.token2idx = {
```

```
        self.PAD_TOKEN: self.PAD_IDX,
```

```
        self.UNK_TOKEN: self.UNK_IDX,
```

```
        self.SOS_TOKEN: self.SOS_IDX,
```

```
        self.EOS_TOKEN: self.EOS_IDX
```

```
    }
```



```
self.idx2token = {v: k for k, v in
self.token2idx.items()}
```

```
def build_vocab(self, tokenized_sentences):
    """Xây dựng vocabulary từ các câu đã tokenize"""
    counter = Counter()
    for tokens in tokenized_sentences:
        counter.update(tokens)

    # Lấy các token phổ biến nhất (trừ các token đặc
    biệt)
    most_common = counter.most_common(self.max_size -
4)

    # Thêm vào vocabulary
    for token, _ in most_common:
        if token not in self.token2idx:
            idx = len(self.token2idx)
            self.token2idx[token] = idx
            self.idx2token[idx] = token

    print(f'Kích thước vocabulary:
{len(self.token2idx)}')

def encode(self, tokens):
    """Chuyển đổi tokens thành indices"""
    return [self.token2idx.get(token, self.UNK_IDX) for
token in tokens]
```

```

def decode(self, indices):
    """Chuyển đổi indices thành tokens"""
    return [self.idx2token.get(idx, self.UNK_TOKEN) for
idx in indices]

```

```

def __len__(self):
    return len(self.token2idx)

```

Phụ lục E: Lớp Dataset Tùy chỉnh

Đây là định nghĩa lớp TranslationDataset kế thừa từ Dataset của PyTorch, cho phép lưu trữ cặp câu (source, target). Phương thức `__getitem__` thực hiện tokenize, cắt ngắn độ dài, thêm token <sos> và <eos>, rồi chuyển các token này thành chỉ số Tensor.

```

class TranslationDataset(Dataset):
    """Custom Dataset cho các cặp câu dịch"""

    def __init__(self, data, src_vocab, trg_vocab,
src_tokenizer, trg_tokenizer, max_len=50):
        self.data = data
        self.src_vocab = src_vocab
        self.trg_vocab = trg_vocab
        self.src_tokenizer = src_tokenizer
        self.trg_tokenizer = trg_tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        src_text, trg_text = self.data[idx]

```

```

# Tokenize
src_tokens = self.src_tokenizer(src_text)
trg_tokens = self.trg_tokenizer(trg_text)

# Giới hạn độ dài
src_tokens = src_tokens[:self.max_len]
trg_tokens = trg_tokens[:self.max_len]

# Thêm <sos> và <eos> tokens
src_tokens = [self.src_vocab.SOS_TOKEN] +
src_tokens + [self.src_vocab.EOS_TOKEN]
trg_tokens = [self.trg_vocab.SOS_TOKEN] +
trg_tokens + [self.trg_vocab.EOS_TOKEN]

# Chuyển thành indices
src_indices = self.src_vocab.encode(src_tokens)
trg_indices = self.trg_vocab.encode(trg_tokens)

return torch.tensor(src_indices),
torch.tensor(trg_indices)

```

Phụ lục F: Hàm Collate Tùy chỉnh

Hàm `collate_fn` dùng để xử lý batch dữ liệu: sắp xếp batch theo chiều dài của câu nguồn giảm dần, thực hiện padding các câu nguồn và câu đích sao cho có cùng độ dài. Hàm trả về các tensor sau padding cùng độ dài và các độ dài gốc dùng để packing.

```
def collate_fn(batch):
```

```
    """
```

Hàm collate tùy chỉnh để padding và packing:

- Sắp xếp batch theo độ dài giảm dần
- Padding các câu về cùng độ dài

```

"""
# Tách source và target
src_batch, trg_batch = zip(*batch)

# Lấy độ dài
src_lengths = torch.tensor([len(s) for s in src_batch])
trg_lengths = torch.tensor([len(t) for t in trg_batch])

# Sắp xếp theo độ dài source (giảm dần) để
pack_padded_sequence
src_lengths, sort_idx =
src_lengths.sort(descending=True)
src_batch = [src_batch[i] for i in sort_idx]
trg_batch = [trg_batch[i] for i in sort_idx]
trg_lengths = trg_lengths[sort_idx]

# Padding sequences - format [seq_len, batch_size]
src_padded = pad_sequence(src_batch, batch_first=False,
padding_value=0)
trg_padded = pad_sequence(trg_batch, batch_first=False,
padding_value=0)

return src_padded, trg_padded, src_lengths, trg_lengths

```

Phụ lục G: Lớp Encoder

Định nghĩa lớp Encoder của mô hình seq2seq. Encoder dùng embedding chuyển mỗi chỉ số từ thành vector, sau đó dùng LSTM để xử lý chuỗi. Phương thức forward trả về các trạng thái (hidden, cell) cuối cùng của LSTM, đóng vai trò là *context vector* cho Decoder.

```

class Encoder(nn.Module):
    """Encoder: Mã hóa câu nguồn thành context vector"""

```

```

def __init__(self, input_dim, emb_dim, hid_dim,
n_layers, dropout):
    super().__init__()
    self.hid_dim = hid_dim
    self.n_layers = n_layers

    # Lớp embedding chuyển index thành vector
    self.embedding = nn.Embedding(input_dim, emb_dim)
    # LSTM để xử lý chuỗi
    self.rnn = nn.LSTM(emb_dim, hid_dim, n_layers,
dropout=dropout)
    self.dropout = nn.Dropout(dropout)

def forward(self, src, src_len):
    # src: [src_len, batch_size]
    # src_len: [batch_size]

    # Embedding và dropout
    embedded = self.dropout(self.embedding(src)) #
[src_len, batch_size, emb_dim]

    # Pack padded sequence để bỏ qua các vị trí padding
    packed_embedded = pack_padded_sequence(embedded,
src_len.cpu(), enforce_sorted=True)

    # Chạy qua LSTM
    packed_outputs, (hidden, cell) =
self.rnn(packed_embedded)

```

```

        # hidden: [n_layers, batch_size, hid_dim] - context
vector

        # cell: [n_layers, batch_size, hid_dim] - cell state

    return hidden, cell

```

Phụ lục H: Lớp Decoder

Định nghĩa lớp Decoder. Decoder cũng sử dụng embedding và LSTM, nhưng có thêm một lớp fully-connected fc_out để dự đoán từ kế tiếp. Phương thức forward nhận đầu vào là từ hiện tại (một chỉ số), cũng như trạng thái hidden, cell từ encoder, rồi trả về xác suất của từ tiếp theo và trạng thái mới.

```

class Decoder(nn.Module):
    """Decoder: Giải mã context vector thành câu đích"""
    def __init__(self, output_dim, emb_dim, hid_dim,
n_layers, dropout):
        super().__init__()
        self.output_dim = output_dim
        self.hid_dim = hid_dim
        self.n_layers = n_layers

        # Lớp embedding cho câu đích
        self.embedding = nn.Embedding(output_dim, emb_dim)
        # LSTM để sinh từng từ
        self.rnn = nn.LSTM(emb_dim, hid_dim, n_layers,
dropout=dropout)
        # Lớp fully connected để dự đoán từ tiếp theo
        self.fc_out = nn.Linear(hid_dim, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, input, hidden, cell):
        # input: [batch_size] - từ hiện tại

```

```

        # hidden: [n_layers, batch_size, hid_dim]
        # cell: [n_layers, batch_size, hid_dim]

        # Thêm chiều sequence length
        input = input.unsqueeze(0) # [1, batch_size]

        # Embedding và dropout
        embedded = self.dropout(self.embedding(input)) #
[1, batch_size, emb_dim]

        # Chạy qua LSTM với context từ encoder
        output, (hidden, cell) = self.rnn(embedded, (hidden,
cell))

        # output: [1, batch_size, hid_dim]

        # Dự đoán từ tiếp theo
        prediction = self.fc_out(output.squeeze(0)) #
[batch_size, output_dim]

        return prediction, hidden, cell

```

Phụ lục I: Mô hình Seq2Seq

Lớp Seq2Seq kết hợp Encoder và Decoder. Phương thức forward thực hiện encoder cho câu nguồn, rồi lặp qua từng bước thời gian để decoder sinh các từ đích, sử dụng cơ chế *Teacher Forcing* (với xác suất được điều chỉnh).

```

class Seq2Seq(nn.Module):
    """Mô hình Seq2Seq kết hợp Encoder và Decoder"""
    def __init__(self, encoder, decoder, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

```

```

        self.device = device

        # Kiểm tra Encoder và Decoder phải có cùng hidden_dim
và n_layers
        assert encoder.hid_dim == decoder.hid_dim
        assert encoder.n_layers == decoder.n_layers

    def forward(self, src, src_len, trg,
teacher_forcing_ratio=0.5):
        # src: [src_len, batch_size]
        # trg: [trg_len, batch_size]
        # teacher_forcing_ratio: xác suất sử dụng từ đúng
thay vì từ dự đoán

        batch_size = src.shape[1]
        trg_len = trg.shape[0]
        trg_vocab_size = self.decoder.output_dim

        # Tensor lưu kết quả dự đoán
        outputs = torch.zeros(trg_len, batch_size,
trg_vocab_size).to(self.device)

        # Encoder mã hóa câu nguồn
        hidden, cell = self.encoder(src, src_len)

        # Từ đầu tiên là <sos>
        input = trg[0, :]

        # Decoder sinh từng từ

```



```

        for t in range(1, trg_len):
            output, hidden, cell = self.decoder(input,
hidden, cell)

            outputs[t] = output

            # Teacher Forcing: 50% dùng từ đúng, 50% dùng từ
dự đoán

            best_guess = output.argmax(1)
            input = trg[t] if random.random() <
teacher_forcing_ratio else best_guess

        return outputs

```

Phụ lục J: Đọc dữ liệu và xây dựng Vocabulary

Đoạn mã này định nghĩa hàm `load_data` để đọc cặp câu từ file (mỗi dòng chứa một câu), rồi đọc tập train, validation, test. Sau đó in kích thước các tập và một ví dụ. Tiếp theo, xây dựng hai vocabulary cho tiếng Anh và tiếng Pháp bằng cách tokenizing và dùng `Vocabulary.build_vocab`.

```

# ===== ĐỌC DỮ LIỆU =====

def load_data(src_file, trg_file):
    """Đọc file dữ liệu song ng English-French"""
    with open(src_file, 'r', encoding='utf-8') as f:
        src_sentences = [line.strip() for line in f]

    with open(trg_file, 'r', encoding='utf-8') as f:
        trg_sentences = [line.strip() for line in f]

    return list(zip(src_sentences, trg_sentences))

# Đọc dữ liệu train, validation và test

```

```

train_data      =      load_data('../dataset/train.en',
                                  '../dataset/train.fr')
val_data        =      load_data('../dataset/val.en',
                                  '../dataset/val.fr')
test_data       =      load_data('../dataset/test.en',
                                  '../dataset/test.fr')

print(f'Tập train: {len(train_data)} cặp câu')
print(f'Tập validation: {len(val_data)} cặp câu')
print(f'Tập test: {len(test_data)} cặp câu')
print(f'\nVí dụ:')
print(f'EN: {train_data[0][0]}')
print(f'FR: {train_data[0][1]}')

# ===== XÂY DỰNG VOCABULARY =====
print('\nXây dựng vocabulary tiếng Anh...')
SRC_vocab = Vocabulary(max_size=10000)
src_tokenized = [tokenize_en(src) for src, trg in
train_data]
SRC_vocab.build_vocab(src_tokenized)

print('\nXây dựng vocabulary tiếng Pháp...')
TRG_vocab = Vocabulary(max_size=10000)
trg_tokenized = [tokenize_fr(trg) for src, trg in
train_data]
TRG_vocab.build_vocab(trg_tokenized)

```

Phụ lục K: Tạo DataLoader

Sau khi có dataset và vocabulary, mã dưới đây tạo các đối tượng Dataset và DataLoader cho tập train, validation, test. Kích thước batch được đặt, sử dụng

hàm `collate_fn` đã định nghĩa để xử lý padding. Cuối cùng in ra số batch của mỗi tập.

```
# Tạo datasets
```

```
train_dataset = TranslationDataset(train_data, SRC_vocab,  
TRG_vocab, tokenize_en, tokenize_fr)
```

```
valid_dataset = TranslationDataset(val_data, SRC_vocab,  
TRG_vocab, tokenize_en, tokenize_fr)
```

```
test_dataset = TranslationDataset(test_data, SRC_vocab,  
TRG_vocab, tokenize_en, tokenize_fr)
```

```
# Tạo dataloaders
```

```
BATCH_SIZE = 128
```

```
train_iterator = DataLoader(train_dataset,  
batch_size=BATCH_SIZE, shuffle=True,  
collate_fn=collate_fn,  
num_workers=0)
```

```
valid_iterator = DataLoader(valid_dataset,  
batch_size=BATCH_SIZE, shuffle=False,  
collate_fn=collate_fn,  
num_workers=0)
```

```
test_iterator = DataLoader(test_dataset,  
batch_size=BATCH_SIZE, shuffle=False,  
collate_fn=collate_fn,  
num_workers=0)
```

```
print(f'Số batch train: {len(train_iterator)}')
```

```
print(f'Số batch validation: {len(valid_iterator)}')
```

```
print(f'Số batch test: {len(test_iterator)}')
```

Phụ lục L: Khởi tạo mô hình

Đoạn mã thiết lập các siêu tham số của mô hình (kích thước vocabulary, kích thước embedding, số chiều ẩn, số lớp, dropout), sau đó khởi tạo đối tượng Encoder, Decoder và Seq2Seq. Hàm `init_weights` khởi tạo trọng số ngẫu nhiên cho các tham số trong khoảng $[-0.08, 0.08]$. Cuối cùng in tổng số tham số của mô hình.

```
# ===== THIẾT LẬP CÁC THAM SỐ MÔ HÌNH =====  
INPUT_DIM = len(SRC_vocab)      # Kích thước vocabulary nguồn  
OUTPUT_DIM = len(TRG_vocab)     # Kích thước vocabulary đích  
ENC_EMB_DIM = 256               # Số chiều embedding cho  
encoder  
DEC_EMB_DIM = 256               # Số chiều embedding cho  
decoder  
HID_DIM = 512                   # Số chiều hidden state  
(theo recommended parameters)  
N_LAYERS = 2                    # Số lớp LSTM  
ENC_DROPOUT = 0.5               # Dropout cho encoder  
DEC_DROPOUT = 0.5               # Dropout cho decoder  
  
# Khởi tạo Encoder, Decoder và Seq2Seq model  
enc = Encoder(INPUT_DIM, ENC_EMB_DIM, HID_DIM, N_LAYERS,  
ENC_DROPOUT)  
dec = Decoder(OUTPUT_DIM, DEC_EMB_DIM, HID_DIM, N_LAYERS,  
DEC_DROPOUT)  
model = Seq2Seq(enc, dec, device).to(device)  
  
# Khởi tạo trọng số ngẫu nhiên trong khoảng  $[-0.08, 0.08]$   
def init_weights(m):  
    for name, param in m.named_parameters():  
        nn.init.uniform_(param.data, -0.08, 0.08)
```

```
model.apply(init_weights)
```

```
print(f"Model parameters: {sum(p.numel() for p in  
model.parameters() if p.requires_grad):,}")
```

Phụ lục M: Optimizer và Loss

Đây là đoạn mã thiết lập optimizer (Adam) và hàm mất mát (CrossEntropyLoss). Chỉ số padding (TRG_PAD_IDX) được thêm vào để hàm loss bỏ qua các token padding không đóng góp vào tính toán loss.

```
# ===== OPTIMIZER VÀ LOSS FUNCTION =====  
LEARNING_RATE = 0.001  
optimizer = optim.Adam(model.parameters(),  
lr=LEARNING_RATE) # Optimizer Adam  
TRG_PAD_IDX = TRG_vocab.PAD_IDX  
criterion = nn.CrossEntropyLoss(ignore_index=TRG_PAD_IDX)  
# Bỏ qua loss của padding token
```

Phụ lục N: Hàm huấn luyện và đánh giá

Hai hàm quan trọng train và evaluate được định nghĩa ở đây. Hàm train huấn luyện mô hình trong một epoch trên tập training, thực hiện forward/backward pass và cập nhật trọng số. Hàm evaluate đánh giá mô hình trên tập validation mà không dùng teacher forcing.

```
def train(model, iterator, optimizer, criterion, clip):  
    """Hàm huấn luyện model cho 1 epoch"""  
    model.train() # Chuyển sang chế độ training  
    epoch_loss = 0  
  
    for src, trg, src_len, trg_len in iterator:  
        src, trg = src.to(device), trg.to(device)  
  
        # Reset gradient  
        optimizer.zero_grad()
```

```

    # Forward pass
    output = model(src, src_len, trg)

    # Reshape output và target để tính loss
    # Bỏ <sos> token (vị trí đầu tiên)
    output_dim = output.shape[-1]
    output = output[1:].view(-1, output_dim)      #
    [(trg_len - 1) * batch_size, output_dim]
    trg = trg[1:].view(-1)                        # [(trg_len
    - 1) * batch_size]

    # Tính loss
    loss = criterion(output, trg)

    # Backward pass
    loss.backward()

    # Clip gradient để tránh exploding gradient
    torch.nn.utils.clip_grad_norm_(model.parameters(),
clip)

    # Cập nhật trọng số
    optimizer.step()
    epoch_loss += loss.item()

    return epoch_loss / len(iterator)

def evaluate(model, iterator, criterion):

```

```

"""Hàm đánh giá model trên validation set"""
model.eval() # Chuyển sang chế độ evaluation
epoch_loss = 0

with torch.no_grad(): # Không tính gradient
    for src, trg, src_len, trg_len in iterator:
        src, trg = src.to(device), trg.to(device)

        # Forward pass, không dùng teacher forcing
        (ratio=0)
        output = model(src, src_len, trg,
teacher_forcing_ratio=0)

        # Reshape để tính loss
        output_dim = output.shape[-1]
        output = output[1:].view(-1, output_dim)
        trg = trg[1:].view(-1)

        loss = criterion(output, trg)
        epoch_loss += loss.item()

    return epoch_loss / len(iterator)

```

Phụ lục O: Vòng lặp huấn luyện chính

Đây là đoạn mã chạy vòng lặp huấn luyện đầy đủ qua nhiều epoch. Các tham số như số epoch tối đa, gradient clipping, early stopping (PATIENCE) được thiết lập. Trong mỗi epoch, chạy hàm train trên tập train và hàm evaluate trên tập validation, lưu lại loss và in ra thông tin. Mô hình tốt nhất (theo loss validation) được lưu vào file 'best_model.pth'.

```

# Thiết lập tham số training
N_EPOCHS = 10 # Số epoch tối đa

```

```
CLIP = 1 # Gradient clipping để tránh exploding gradient
best_valid_loss = float('inf') # Lưu loss nhỏ nhất trên
validation set
early_stop_cnt = 0 # Đếm số epoch không cải thiện
PATIENCE = 3 # Dừng sớm nếu không cải thiện sau 3 epoch

# Lưu loss để vẽ biểu đồ
train_losses = []
valid_losses = []

# Vòng lặp training
for epoch in range(N_EPOCHS):
    start_time = time.time()

    # Train model trên tập train
    train_loss = train(model, train_iterator, optimizer,
criterion, CLIP)
    # Đánh giá model trên tập validation
    valid_loss = evaluate(model, valid_iterator, criterion)

    end_time = time.time()

    # Lưu loss
    train_losses.append(train_loss)
    valid_losses.append(valid_loss)

    # In kết quả epoch
    print(f'Epoch: {epoch+1:02} | Time: {end_time -
start_time:.2f}s')
```



```

        print(f'\tTrain Loss: {train_loss:.3f} | Train PPL:
{math.exp(train_loss):7.3f}')
        print(f'\t Val. Loss: {valid_loss:.3f} | Val. PPL:
{math.exp(valid_loss):7.3f}')

# Lưu model tốt nhất
if valid_loss < best_valid_loss:
    best_valid_loss = valid_loss
    torch.save(model.state_dict(), 'best_model.pth')
    print('\t>>> Saved best model!')
    early_stop_cnt = 0 # Reset bộ đếm
else:
    early_stop_cnt += 1 # Tăng bộ đếm nếu không cải
thiện
    if early_stop_cnt >= PATIENCE:
        print("\nEarly Stopping!") # Dừng sớm nếu đủ
patience
        break

```

Phụ lục P: Vẽ biểu đồ Loss

Sau khi huấn luyện, mã này vẽ đồ thị thể hiện tiến trình loss của tập huấn luyện và tập validation theo các epoch. Đồ thị được lưu vào file 'loss_chart.png', đồng thời lưu mảng train_losses và valid_losses thành file .npy để có thể dùng sau này.

Vẽ biểu đồ training và validation loss

```

plt.figure(figsize=(10, 6))
plt.plot(train_losses, label='Training Loss', marker='o')
plt.plot(valid_losses, label='Validation Loss', marker='s')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training và Validation Loss theo Epochs')
plt.legend()

```

```
plt.grid(True)
plt.savefig('loss_chart.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
print('Đã lưu biểu đồ loss vào loss_chart.png')
```

```
# Lưu losses để sử dụng sau này
np.save('train_losses.npy', np.array(train_losses))
np.save('val_losses.npy', np.array(valid_losses))
```

Phụ lục Q: Hàm dịch câu

Hai hàm `translate_sentence` và `translate` được định nghĩa để dịch một câu riêng lẻ. `translate_sentence` tokenizes câu nguồn, thêm token `<sos>`, `<eos>`, đưa vào encoder và sau đó từng bước sử dụng decoder để sinh từ dự đoán cho đến khi gặp `<eos>` hoặc đến độ dài tối đa. `translate` gọi `translate_sentence` và nối các token thành chuỗi kết quả.

```
def translate_sentence(sentence, src_vocab, trg_vocab,
src_tokenizer, model, device, max_len=50):
```

```
    """
```

Hàm dịch 1 câu từ nguồn sang đích

Args:

sentence: Câu cần dịch (string hoặc list tokens)

max_len: Độ dài tối đa của câu dịch

Returns:

List các token đã dịch

```
    """
```

```
    model.eval()
```

```
    # Tokenize câu nguồn
```

```
    if isinstance(sentence, str):
```

```

        tokens = [token.lower() for token in
src_tokenizer(sentence)]
    else:
        tokens = [token.lower() for token in sentence]

    # Thêm <sos> và <eos>
    tokens = ['<sos>'] + tokens + ['<eos>']

    # Chuyển thành indices và tensor
    src_indexes = src_vocab.encode(tokens)
    src_tensor =
torch.LongTensor(src_indexes).unsqueeze(1).to(device) #
[src_len, 1]

    src_len = torch.LongTensor([len(src_indexes)])

    # Encode câu nguồn
    with torch.no_grad():
        hidden, cell = model.encoder(src_tensor, src_len)

    # Bắt đầu decode từ <sos>
    trg_indexes = [trg_vocab.SOS_IDX]

    # Sinh từng từ cho đến khi gặp <eos> hoặc đạt max_len
    for i in range(max_len):
        trg_tensor = torch.LongTensor([trg_indexes[-
1]]).to(device)

        with torch.no_grad():

```

```

        output, hidden, cell =
model.decoder(trg_tensor, hidden, cell)

    # Lấy từ có xác suất cao nhất
    pred_token = output.argmax(1).item()
    trg_indexes.append(pred_token)

    # Dừng nếu gặp <eos>
    if pred_token == trg_vocab.EOS_IDX:
        break

    # Chuyển indices thành tokens (bỏ <sos> và <eos>)
    trg_tokens = [trg_vocab.idx2token[i] for i in
trg_indexes]

    return trg_tokens[1:-1]

def translate(sentence: str) -> str:
    """Hàm dịch theo yêu cầu đề bài - trả về chuỗi đã dịch"""
    pred_tokens = translate_sentence(sentence, SRC_vocab,
TRG_vocab, tokenize_en, model, device)
    return ' '.join(pred_tokens)

```

Phụ lục R: Kiểm thử dịch

Đoạn mã này tải mô hình tốt nhất từ file 'best_model.pth' và thử dịch một số câu ví dụ tiếng Anh. Kết quả hiển thị câu gốc (EN) và bản dịch (FR) in ra.

```

# ===== LOAD MODEL TỐT NHẤT VÀ TEST DỊCH =====
model.load_state_dict(torch.load('best_model.pth'))

test_sentences = [
    "I love programming",

```

```

    "A man is standing on a ladder",
    "Two dogs are running in the park"
]

```

```

for sent in test_sentences:
    translation = translate(sent)
    print(f"EN: {sent}")
    print(f"FR: {translation}\n")

```

Phụ lục S: Tính BLEU Score

Đoạn mã dưới đây tính BLEU score trên tập Test để đánh giá chất lượng dịch của mô hình. Hàm `calculate_bleu` lặp qua tối đa `max_samples` câu của tập cho trước, dịch từng câu và so sánh với câu tham chiếu để tính BLEU của cả tập. Cuối cùng in ra BLEU score theo tỷ lệ phần trăm.

```

from nltk.translate.bleu_score import sentence_bleu,
corpus_bleu
import nltk
nltk.download('punkt', quiet=True)

def calculate_bleu(model, dataset, src_vocab, trg_vocab,
src_tokenizer, trg_tokenizer, device, max_samples=1000):
    """
    Tính BLEU score trên dataset
    BLEU đánh giá chất lượng dịch bằng cách so sánh với câu
    tham chiếu
    """
    model.eval()
    references = [] # Câu tham chiếu (ground truth)
    hypotheses = [] # Câu dự đoán (model prediction)

    num_samples = min(len(dataset), max_samples)

```

```

for i in range(num_samples):
    src_sent, trg_sent = dataset.data[i]

    # Tokenize câu tham chiếu và dịch câu nguồn
    trg_tokens = trg_tokenizer(trg_sent)
    pred_tokens = translate_sentence(src_sent,
src_vocab, trg_vocab, src_tokenizer, model, device)

    references.append([trg_tokens])
    hypotheses.append(pred_tokens)

# Tính BLEU score trên toàn bộ corpus
bleu_score = corpus_bleu(references, hypotheses) * 100
return bleu_score

# Load test dataset (dùng test_data đã có từ phần 10)
test_dataset_for_bleu = TranslationDataset(test_data,
SRC_vocab, TRG_vocab,
tokenize_en, tokenize_fr)

bleu_score = calculate_bleu(model, test_dataset_for_bleu,
SRC_vocab,
TRG_vocab, tokenize_en,
tokenize_fr, device,
max_samples=1000)

print(f"\nBLEU SCORE (TEST SET): {bleu_score:.2f}")

```

Phụ lục T: Phân tích lỗi (5 ví dụ từ tập Test)

Cuối cùng, đoạn mã này in ra chi tiết 5 ví dụ dịch từ tập Test. Với mỗi câu, in ra câu nguồn, câu tham chiếu, kết quả dịch của mô hình và BLEU của câu đó. Đồng thời hiển thị chi tiết *debug* các token tham chiếu và dự đoán, đếm số từ trùng khớp, và phân tích lỗi gồm: khác biệt độ dài, từ OOV, từ thiếu/thừa, và nguyên nhân gọi ý.

```
# ===== PHÂN TÍCH LỖI CHI TIẾT =====

print("\n" + "="*70)
print("PHÂN TÍCH LỖI - 5 VÍ DỤ TỪ TEST SET")
print("="*70)

# Chọn 5 ví dụ cố định từ test set
sample_indices = [0, 1, 2, 3, 4]

for idx, i in enumerate(sample_indices, 1):
    src_sent, trg_sent = test_data[i]

    # Dịch câu và tokenize reference đúng cách
    pred_tokens = translate_sentence(src_sent, SRC_vocab,
    TRG_vocab, tokenize_en, model, device)
    pred_sent = ' '.join(pred_tokens)

    # Tokenize reference giống như cách model đã học
    (lowercase)
    trg_tokens = [tok.lower() for tok in
    tokenize_fr(trg_sent)]

    # Tính BLEU score - sử dụng smoothing để tránh BLEU = 0
    from nltk.translate.bleu_score import SmoothingFunction
    smoothie = SmoothingFunction().method1
```

```
bleu = sentence_bleu([trg_tokens], pred_tokens,
smoothing_function=smoothie) * 100
```

```
print(f"\n{'='*70}")
print(f"VÍ DỤ {idx}")
print(f"{'='*70}")
print(f"Source (EN): {src_sent}")
print(f"Reference (FR): {trg_sent}")
print(f"Prediction: {pred_sent}")
print(f"BLEU: {bleu:.2f}")
```

```
# Debug: In ra tokens để kiểm tra
print(f"\nDEBUG:")
print(f"Reference tokens: {trg_tokens[:10]}...")
print(f"Predicted tokens: {pred_tokens[:10]}...")
```

```
# Đếm số từ trùng khớp
matched = len(set(trg_tokens) & set(pred_tokens))
print(f"Số từ trùng khớp:
{matched}/{len(set(trg_tokens))}")
```

```
# ===== PHÂN TÍCH CÁC LOẠI LỖI =====
print(f"\nPHÂN TÍCH LỖI:")
```

```
# 1. Phân tích độ dài câu
src_len = len(src_sent.split())
ref_len = len(trg_sent.split())
pred_len = len(pred_sent.split())
```



```
print(f" Độ dài: Source={src_len}, Reference={ref_len},  
Prediction={pred_len}")
```

```
if abs(pred_len - ref_len) > 3:  
    print(f" → Lỗi độ dài: Chênh lệch {abs(pred_len -  
ref_len)} từ")
```

```
# 2. Kiểm tra từ OOV (Out-Of-Vocabulary)  
src_tokens = tokenize_en(src_sent)  
oov_words = [word for word in src_tokens if  
SRC_vocab.token2idx.get(word, SRC_vocab.UNK_IDX) ==  
SRC_vocab.UNK_IDX]  
if oov_words:  
    print(f" → Từ OOV: {oov_words}")  
    print(f"          Nguyên nhân: Từ không có trong  
vocabulary")
```

```
# 3. So sánh tokens: từ thiếu và từ thừa  
missing = set(trg_tokens) - set(pred_tokens)  
extra = set(pred_tokens) - set(trg_tokens)  
if missing:  
    print(f" → Từ thiếu: {list(missing)[:5]}")  
if extra:  
    print(f" → Từ thừa: {list(extra)[:5]}")
```

```
# 4. Phân tích nguyên nhân và đề xuất giải pháp  
print(f"\n Nguyên nhân:")  
if matched == 0:
```

```

        print(f"        - KHÔNG CÓ TỪ NÀO TRÙNG KHỚP → Model
dịch hoàn toàn sai")
        print(f"        - Có thể model chưa được train đủ hoặc
quá phức tạp cho dữ liệu")
        elif matched < 3:
            print(f"        - Rất ít từ trùng khớp → Dịch sai nghiêm
trọng")
        if src_len > 20:
            print(f"        - Câu dài ({src_len} từ) → Context vector
bottleneck")
            print(f"        Model khó nén toàn bộ thông tin câu
dài vào 1 vector")
            if oov_words:
                print(f"        - Có từ OOV → Model không học được các
từ này")
                print(f"        Giải pháp: Tăng kích thước vocabulary
hoặc dùng subword tokenization")
            if bleu < 30:
                print(f"        - BLEU thấp → Dịch sai cấu trúc hoặc ngữ
pháp")
                print(f"        - Giải pháp chính: Thêm Attention
Mechanism")
                print(f"        + Attention giúp Decoder tập trung vào
từng phần của câu nguồn")
                print(f"        + Giải quyết vấn đề context vector
bottleneck")

```